

Takarda Decision Dossier

Architecture and decision record for the Takarda results-checking platform, prepared for the National Secondary Certificate Council contract.

Contents

Part A.1 - Business Requirements Document
Part A.2 - Product Requirements Document
Part A.3 - Functional Requirements Document
Part A.4 - Work Breakdown Structure
Part B.0 - The Stakeholder Conflicts
Part B.1 - Quality Attributes, Ranked
Part B.2 - C4 Diagrams
Part B.3 - The Architecture Pattern
Part B.4 - Architecture Decision Records
Part B.5 - The Trade-off Register
Part B.6 - What You Are Not Building
Part C.1 - Protocol Choice Per Consumer
Part C.2 - Transport Security
Part C.3 - The API Contract
Part C.4 - The PIN Purchase, Fixed
Part D.1 - The Data Model
Part D.2 - The Database Architecture
Part D.3 - Five Queries, Predicted
Part D.4 - What Every Index Costs
Part D.5 - What Would Prove You Wrong
Part E - How You Worked With AI
Appendix - Stated Assumptions

Part A.1 — Business Requirements Document

The problem

The Council releases one national examination's results once a year and answers verification requests from employers and universities year-round. Today this runs on a 2009 system with no institutional memory attached to it. That system already fails its users in a measurable way: **9,100 candidates were charged twice** for their result-checker PIN last year, with no automated way to detect or refund the duplicate. It also has no stated capacity story for the concentrated demand of results day — 1,400,000 checks in the first hour, peaking near 2,900 a second — against the eleven-month baseline of under 20 a second.

Three groups suffer under the current arrangement. **Candidates**, some on 2G with feature phones, risk being charged twice for a result they cannot get through to check on the day it matters most. **Employers and universities**, whose verification volume is contracted to grow twelve-fold over the coming year (40,000 to 500,000 a month), need a channel that can absorb that growth without a matching twelve-fold cost. **The Council itself** carries regulatory exposure: it must be able to reproduce any released result exactly as it stood on any past date for fifty years, and trace every amendment to the officer who made it — a guarantee a 2009 system with no documented architecture cannot be assumed to provide today.

What success looks like

Success is checkable, not aspirational:

- The result-check endpoint answers **99.95% of requests during the results-day peak hour** (≤ 22 seconds of failure in that hour).
- Infrastructure spend does not exceed **NGN 14,000,000/month**, including on results day.
- The **9,100-a-year double-charge** figure falls to a number detected and refunded automatically, with zero requiring a human to read a spreadsheet.
- Every amendment carries an officer identity and evidence reference, retrievable on request, for the life of the fifty-year retention window.

Options considered

1. **Do nothing** — keep the 2009 system, absorb the known double-charge cost and the unmeasured results-day risk. Rejected: the double-charge problem is already quantified and costing candidates money on every sitting, and the employers' association growth (12x within a year) has no ceiling on the current system's demonstrated capacity.
2. **Rebuild everything as one always-on, horizontally scaled service** sized to the results-day peak year-round. Rejected: this is the option that violates the Finance Director's flat NGN 14,000,000/month ceiling directly — provisioning for a 150x spike continuously means paying for capacity that sits idle for all but roughly one hour a year.
3. **Recommended: pre-materialize results-day reads, keep a modestly-provisioned transactional core for everything else.** Serve the results-day spike from a flat-cost static/cache tier built before noon, and keep a normally-provisioned relational database for the rest of the year's real workload (verification, school portal access outside results day, amendments, officer search). Full reasoning and rejected architecture patterns are in Part B.

What the Council is committing to

- **Spend:** infrastructure at the current NGN 14,000,000/month ceiling; no incremental spend proportional to results-day traffic or the employers' association's growth to 500,000 verifications/month.
- **Risk accepted:** an amendment made close to results day will not appear on an already-generated results-day payload until a controlled cache-refresh step runs — see Part B, "What you are not building." The Council accepts a bounded propagation delay for amendments in exchange for a flat-cost, always-fast candidate experience.
- **Risk not accepted:** the fifty-year reproducibility and officer-attributed amendment trail are treated as non-negotiable; every architectural decision that touches results data is constrained by that requirement first.
- **Timeline:** first release live in time for the May sitting (nine months from today), built by a team of six engineers including the author. The Work Breakdown Structure (Part A.4) states explicitly what is cut if that does not fit.

Part A.2 — Product Requirements Document

Takarda has one product-level tension that every feature below inherits: it must feel instant to seven very different users at once (a candidate on a feature phone, a school pulling hundreds of rows, an employer's automated system) while costing the same every month regardless of how many of them show up on the same day. Nothing in this document should be read as satisfying every user's first-choice request — Part B.0 names where that isn't possible and what was chosen instead. What follows is what actually ships, for whom, and in what order.

Users and their jobs

User	Job to be done	Good outcome
Candidate (browser)	Check my result the moment it is released	Result appears in one attempt, in seconds, without losing a PIN use to a failed attempt
Candidate (feature phone, 2G, USSD/SMS)	Check my result without a smartphone or good signal	Same result, same PIN, over USSD/SMS, without needing a data connection
Candidate (any channel)	Buy a PIN without being charged twice	One PIN, one charge, confirmed or safely retried
School administrator	Get every result for my school's candidates at noon on results day	A complete, correct file for my ~400 candidates, ready at noon
Employer / university (via the employers' association)	Confirm a certificate is genuine	A yes/no plus the minimal data needed to confirm it, nothing else about the candidate
Employer / university (recurring)	Find out if a certificate I verified before still stands	A status check they can run any time, and a notification if one they checked recently is superseded, without Takarda needing to have their contact details in advance
Council officer	Find a candidate from a partial surname; record an amendment with evidence	Search returns in an interactive time; amendment is recorded with officer and evidence attached, permanently
Regulator (Council board, via the officer/audit path)	Prove what a result said on any past date	Point-in-time reconstruction available for any released result, for 50 years

Scope — Release 1 (the May sitting)

In scope:

- Candidate result check: browser (HTTPS) and USSD/SMS (via telecom aggregator).
- PIN purchase with the reserve/confirm idempotency fix (Part C.4) — the double-charge problem is fixed in Release 1, not deferred.
- Results-day pre-materialized read path for candidates and schools (Part B, ADR-001).
- School bulk export for one sitting.
- Employer/university certificate verification, DPO-filtered.
- Certificate status check and supersession notification (Part B.0, Conflict C's resolution).

- Amendment workflow with officer attribution and evidence reference.
- Officer partial-surname search.
- Core data model supporting 50-year point-in-time reproducibility.

Explicitly out of scope for Release 1 (see Part B.6, "What you are not building," for the full reasoning):

- Real-time push notification of amendments to candidates/schools/employers.
- Self-service live analytics dashboards for the Council beyond the existing per-sitting report.
- Any identity check beyond PIN + exam number for candidate access.
- A self-service dispute/complaints portal beyond the automated double-charge refund.

Release plan and reasoning

Release	Content	Why this order
R0 (months 1–3)	Data model, core write path (results ingestion, amendments), officer search, employer verification API	These have no results-day timing pressure and de-risk the hardest, least reversible decision first — the append-only result model (ADR-002) must be right before real data accumulates under it.
R1 (months 4–7)	PIN purchase + reserve/confirm fix, candidate check (browser + USSD/SMS), results-day read-model builder and cache tier, school bulk export	This is the results-day-critical path and the largest single risk to the May deadline; it is sequenced once the underlying data model (R0) is stable, so the read-model builder has a fixed shape to read from.
R2 (months 8–9)	Load testing against results-day volumes, reconciliation job hardening, cutover runbook, legacy archive migration completion	Final hardening before the May sitting; nothing new is built here — this window exists because the brief's own numbers (150x traffic spike, once a year, no second chance) make a dress rehearsal non-optional.

Metrics that say whether Release 1 worked

- Result-check success rate $\geq 99.95\%$ measured over the results-day peak hour (matches the BRD figure).
- Zero manually-detected double charges; automated detection/refund rate = 100% of true duplicates found in the nightly reconciliation job (measured against the payment partner's settlement file).
- School export files available to all 21,000 schools by 12:00 on results day (not "eventually" — the pre-materialization step must complete before the announced release time).
- 100% of amendments in the sitting carry an officer ID and evidence reference — zero exceptions, since this is a regulatory requirement, not a target.
- Employer verification response contains only Data-Protection-Officer-permitted fields on every response, verified by a fixed test suite of field-leakage checks (out of scope to build, but the FRD specifies exactly what "permitted fields" means so this is testable once built).

Part A.3 — Functional Requirements Document

Every requirement names a trigger and the observable response to that trigger. Numbering is stable across revisions (gaps are left deliberately for later insertion).

PIN purchase (FR-100 series)

FR-101. When a candidate, agent, bank, or the website submits a PIN purchase request with a payment method and an `Idempotency-Key` it has not sent before, the system creates a PIN record in state `PENDING_PAYMENT`, calls the payment partner to charge 350000 kobo, and does not decrement any usage count.

FR-102. When the payment partner confirms the charge succeeded, the system sets the PIN to state `ACTIVE` with `remaining_uses = 5` and returns HTTP 201 with the PIN code.

FR-103. When the payment partner confirms the charge was declined, the system sets the PIN to state `FAILED`, returns HTTP 402, and does not create a usable PIN.

FR-104. When the payment partner does not respond within the configured timeout, the system sets the PIN to state `AWAITING_CONFIRMATION`, returns HTTP 202 with a status-check URL, and takes no further action until the reconciliation job (FR-108) resolves it.

FR-105. When a PIN purchase request arrives with an `Idempotency-Key` that matches an existing record, the system does not call the payment partner again and returns the existing record's current state and outcome.

FR-106. When a PIN purchase request arrives with an `Idempotency-Key` that matches an existing record but a different payment amount or method, the system rejects the request with HTTP 409 and does not attempt a charge.

FR-107. When a candidate is charged twice for the same purchase attempt (detected by the nightly reconciliation job matching the payment partner's settlement file against Takarda's PIN ledger by card token, amount, and a 30-minute proximity window), the system issues an automatic refund via the payment partner's refund API and records the refund against the PIN record, without requiring a human to review the case.

FR-108. When a PIN sits in `AWAITING_CONFIRMATION` for longer than 15 minutes, the reconciliation job queries the payment partner's transaction lookup (or, failing that, the next available settlement file) and transitions the PIN to `ACTIVE` or `FAILED` based on what it finds.

Candidate result check (FR-200 series)

FR-201. When a candidate submits an exam number, sitting identifier, and an active PIN with `remaining_uses > 0`, the system returns the candidate's results for that sitting and decrements `remaining_uses` by one.

FR-202. When a candidate submits a PIN that has already been used five times, the system refuses the check, leaves the PIN's remaining count at zero, and returns the message that the PIN is exhausted.

FR-203. When a candidate submits a PIN and exam number combination where the exam number does not exist for the stated sitting, the system returns HTTP 404 and does not decrement the PIN.

FR-204. When a candidate resubmits a check with the same request `Idempotency-Key` as a prior successful check, the system returns the same stored result and does not decrement `remaining_uses` a

second time.

FR-205. When a result check request arrives during the pre-materialization window before the announced release time, the system returns HTTP 404 regardless of whether the underlying result exists, since no release has occurred yet.

FR-206. When the announced release time is reached, the system serves candidate result checks exclusively from the pre-materialized read tier (Part B, ADR-001), not from a live query against the primary database.

School bulk download (FR-300 series)

FR-301. When an authenticated school requests its result export for a sitting before the pre-materialization job for that sitting has completed, the system returns HTTP 202 with a `Location` header pointing to the eventual file.

FR-302. When an authenticated school requests its result export for a sitting after pre-materialization has completed, the system returns HTTP 200 with a downloadable file containing exactly the candidates enrolled at that school for that sitting.

FR-303. When a school requests results for a candidate not enrolled at that school, the system excludes that candidate from the response; a school never receives another school's candidate.

Verification request (FR-400 series)

FR-401. When the employers' association submits a certificate number and a date of birth that match a candidate's record, the system returns HTTP 200 with only the fields the Data Protection Officer has designated as permitted (certificate validity, sitting, subject grades tied to that certificate) and excludes the candidate's other subjects, address, phone number, and examination centre.

FR-402. When the certificate number does not exist, or exists but the date of birth does not match, the system returns the same generic "no match" response and status code in both cases, so that a caller cannot distinguish a wrong certificate number from a wrong date of birth.

FR-403. When a verification request is received, the system records a verification event (timestamp, requesting account, certificate number, outcome) for audit, regardless of whether the request matched.

FR-404. When the same certificate is verified again after an amendment has changed its underlying result, the verification response reflects the current (post-amendment) value, not the value that was current at the time of any earlier verification.

FR-405. When any party queries `GET /certificates/{certificateNumber}/status` (no date-of-birth challenge required, since this endpoint returns only validity state, never subject grades), the system returns the certificate's current status (`ACTIVE` or `SUPERSEDED`) and, if superseded, the amendment identifier and the date it was superseded.

FR-406. When a certificate's status changes to `SUPERSEDED` as a result of FR-501, the system dispatches a status-change notification to every employer account that verified that certificate within the trailing 12 months, without requiring the employer to re-query first.

Amendment workflow (FR-500 series)

FR-501. When an authenticated Council officer submits an amendment for a result with a reason code and an evidence reference, the system creates a new, immutable result version, links it to the amendment record, the officer's identity, and the prior version, and never overwrites the prior version's stored values.

FR-502. When an amendment is submitted without an evidence reference, the system rejects the request with HTTP 400 and creates no amendment record.

FR-503. When an amendment is recorded, the system marks the result as changed for the next scheduled read-model refresh; it does not attempt to update the results-day static payload already generated for that candidate in real time.

FR-504. When any party requests the value a given result held as of a specific past date, the system reconstructs it from the version history and returns the version that was current on that date, never the latest version, unless the two happen to be the same.

FR-505. When an amendment changes a result that a certificate covers, the system marks that certificate SUPERSEDED, links it to the amendment record, issues a new certificate version for the corrected value, and triggers FR-406's notification — this is the mechanism that gives Conflict C's resolution (Part B.0) a concrete implementation rather than a promise.

USSD / SMS channel (FR-600 series)

FR-601. When a candidate dials the published USSD short code and enters an exam number, sitting code, and PIN through the USSD menu, the system performs the same check as FR-201 through the telecom aggregator gateway and returns the result formatted to fit a single USSD screen (≤ 182 characters per the common USSD page limit).

FR-602. When a candidate's USSD session times out before completing the PIN entry (per the telecom aggregator's session limit), the system does not decrement the PIN's `remaining_uses`, since no check request was completed.

FR-603. When a candidate sends an SMS in the published format (exam number, sitting code, PIN) to the published number, the system replies by SMS with the same result content as the USSD channel, or with a specific error message if the format is invalid.

FR-604. When a candidate on the USSD/SMS channel completes a check successfully, the response contains no data beyond what fits the channel's character limit — grade per subject and nothing else — deferring any richer detail (full transcript, certificate download) to the browser channel.

Part A.4 — Work Breakdown Structure

Team (Assumption 6): 3 backend/platform engineers, 1 DBA, 1 integrations engineer, 1 QA — six including the author. No package exceeds two engineer-weeks; two related sub-week tasks are merged into one row only where doing so does not push that row past the two-week cap. Role/estimate abbreviated: BE=backend, FE=frontend, INT=integrations, DBA, QA, Auth=author.

R0 — Foundation (months 1–3)

#	Deliverable	Deps	Role·Est
1	Data model schema + partitioning design	—	BE+DBA·2w
2	Legacy archive extract tooling	1	BE·2w
3	Legacy transform/validate	2	DBA·2w
4	Legacy load + reconcile	3	DBA·2w
5	Results ingestion service	1	BE·2w
6	Amendment write service	1,5	BE·2w
7	Officer identity integration	—	BE·1w
8	Partial-surname search (trigram)	1,7	BE+DBA·2w
9	Candidate/school ref data loader	1	BE·1w
10	Certificate entity, issuance & response format	1,5	BE·2w
11	Certificate status endpoint, supersession flag & verifier-notification dispatch (FR-405/406/505)	6,10	BE·2w
12	Employer verification API	1,10	BE·2w
13	DPO allow-list sign-off & verification audit logging	12	BE·2w
14	Admin console: search+amend UI	6,8	FE·2w
15	API gateway/v1 scaffold & CI/CD pipeline	—	BE·2w
16	Auth credential issuance: mTLS (employers) + OAuth2 (schools)	—	INT·2w
17	Staging + prod provisioning	—	BE·2w
18	Observability baseline	17	BE·2w
19	Read replicas + lag monitoring	1	DBA·1w
20	Backup / DR strategy	1	DBA·2w
	R0 subtotal		39w

R1 — Results-day critical path (months 4–7)

#	Deliverable	Deps	Role·Est
21	PIN reserve/confirm service	1	BE·2w
22	Payment partner integration	21	INT·2w
23	Payment partner contract tests	22	INT·1w
24	Reconciliation job	21	INT·2w
25	Double-charge detect+refund	24	INT·2w
26	Candidate check API (browser)	5,21	BE·2w
27	USSD gateway integration	26	INT·2w
28	SMS channel integration	26	INT·1w
29	USSD/SMS session handling & content review	27,28	BE·2w
30	Read-model: candidate payloads	5	BE+DBA·2w
31	Read-model: school exports	5,9	BE+DBA·2w
32	Object storage + CDN wiring	30,31	BE·1w
33	Cache refresh on amendment	6,32	BE·2w
34	Rate limiting, public endpoints	26,12	BE·1w
35	Candidate browser check page	26	FE·2w
36	School portal (login+download)	16,31	FE·2w
37	School account provisioning	16	INT·2w
38	School onboarding rollout	37	Auth·1w
39	Employer assoc. integration test	12,16	INT·2w
40	QA: FRD test suite	21–33	QA·2w
41	QA: DPO leakage test suite	12	QA·1w
42	End-to-end integration testing	40,41	QA·2w
	R1 subtotal		38w

R2 — Hardening (months 8–9)

#	Deliverable	Deps	Role·Est
43	Load-test harness (930->2,900/s)	32	QA+BE·2w
44	Load-test execution + tuning	43	BE+DBA·2w
45	Reconciliation hardening & amendment-refresh runbook	24,25,33	INT+BE·2w
46	Legacy migration verification	4	DBA·2w
47	Security review (TLS/mTLS)	16	INT+QA·2w
48	Final FRD regression pass	42	QA·2w
49	Cutover runbook & officer training/UAT	42,44,14	Auth·2w
50	Legacy dual-run support	46	BE·2w
51	Documentation handover	all	Auth·1w
	R2 subtotal		17w

Capacity check

- **Identified work:** 39 + 38 + 17 = **94 engineer-weeks**.
- **Raw capacity:** 6 engineers × 39 weeks (9 months) = **234 engineer-weeks**.
- **Effective capacity (estimate):** raw less ~20% for meetings, review, onboarding, leave = **~187 engineer-weeks**. Labelled estimate — no figure for this exists in the brief.
- **Headroom:** ~187 - 94 ~ **93 engineer-weeks**.

This totals to something a team of six can finish before the May sitting, with headroom to spare — but the headroom is not slack for new scope. It is reserved because packages 22/23, 27/28, 39 depend on organisations Takarda does not control: the payment partner, the telecom aggregator, the employers' association. One of those running long is absorbed by the reserve; two running long at once is not. If that happens, **what gets cut first is scope already marked deferred in Part B.6** — none of the packages above are cuttable without breaking a Regulator, Finance Director, or results-day commitment already made in the BRD.

Part B.0 — The stakeholder conflicts

The brief warns that at least three stakeholder pairs are in direct conflict and one request is impossible as written, and that finding and naming them carries more weight than any diagram in this dossier. They are named here, first, because every later decision — the architecture pattern, the ADRs, the data model — is a response to them, not an independent choice.

Conflict A — Registrar vs. Finance Director

The Registrar wants every candidate served instantly at noon with no queue and no staggered release. The Finance Director's budget is flat at NGN 14,000,000/month regardless of traffic. Results-day load reaches ~2,900 checks/second at peak against an eleven-month baseline under 20/second — a ~150x spike. Serving that spike live against a database provisioned for the average means paying for peak capacity that sits idle almost all year, which is exactly what the Finance Director has ruled out.

Conflict B — Head of Schools vs. Finance Director

The same root cause as Conflict A, arriving from a second direction: 21,000 schools each pulling their full result list at the identical noon instant is a second traffic spike competing for the same flat budget.

Conflict C — Head of Operations vs. Regulator (the impossible request)

The Head of Operations wants an amendment to "appear everywhere immediately, including on certificates an employer has already verified." The Regulator requires every released result to remain reproducible exactly as it stood on any past date, for 50 years, with every amendment traceable to an officer and evidence. Taken literally, Operations' request is impossible on three independent grounds, not just one:

1. **Physical/authority boundary.** Once a verification response has left Takarda's network — printed, screenshotted, filed in an employer's HR system — Takarda has no computational reach over that copy. No API call rewrites a PDF already sitting in someone else's archive.
2. **Legal and evidentiary causality.** What an employer verified on a given date was an accurate statement of the Council's record at that instant. An employment decision may already rest on it. Retroactively mutating that historical fact would break the non-repudiation and evidentiary value the Regulator's 50-year requirement exists to protect in the first place — the two halves of the Head of Operations' own request are in tension with each other, not just with the Regulator.
3. **Distributed-consistency limits.** Even restricted to Takarda's own systems, pushing a change to an unbounded set of external parties with zero-latency, guaranteed delivery and no acknowledgement channel is not achievable without those parties running a persistent connection to Takarda, which they don't.

Conflict D — Chief Executive vs. the combined stakeholder wishlist

The Chief Executive fixes the constraint that decides everything else: go live for the May sitting, nine months away, with six engineers. Taken together, the Registrar, Head of Schools, Head of Operations, Regulator, DPO, and the employers' association's growth target describe a system whose honestly-decomposed scope exceeds what six engineers can build in that time (Part A.4 shows the arithmetic). This is a capacity conflict, not a technical one, and it is resolved the same way the others are: by naming, in advance, which capabilities are cut to make the date (Part B.6), rather than discovering the shortfall in month eight.

Conflict E — Data Protection Officer vs. the default shape of a verification API

The default REST instinct for "let an employer check a certificate" is a resource-read endpoint that returns the candidate record it finds. The DPO's requirement — an employer verifying one certificate must never see the candidate's other subjects, address, phone, or examination centre — is in direct conflict with that default, because a generic read naturally returns the row it finds, not a filtered subset of it. This is resolved by never building the generic endpoint at all: the verification API (Part C.3) is a dedicated resource with an allow-listed response shape from the first line of its design, not a general candidate lookup with redaction bolted on afterward.

The resolution adopted across this dossier

For A and B: results-day reads for candidates and schools are served from a pre-materialized static tier built before noon, not from the live database — see ADR-001. This buys the Registrar and the Head of Schools their "instant, everyone, at noon" experience inside a cost envelope that is flat by construction, because cost there scales with data size, not with request rate.

For C: "everywhere immediately" is honoured for every *future* access — any new check or verification after an amendment sees the current value at once — but not by rewriting a verification response an employer already has in hand. Results are stored as append-only, versioned records (ADR-002), so the Regulator's reproducibility requirement is served by the same version history that makes "current value" well-defined. To give the Head of Operations the closest thing to their actual request that is still honest, every certificate carries a status-check reference (a URI, rendered as a QR code on the printed certificate) that any holder can re-query at any time: `GET /certificates/{certificateNumber}/status`. When an amendment supersedes a certificate, its status flips to `SUPERSEDED`, naming the amendment and the current value, and Takarda dispatches a status-change notification to any employer who verified that certificate in the trailing 12 months (Part C.3, Part A.3 FR-405/FR-406). This does not rewrite what the employer already has — it gives them an active way to find out it changed, which is the strongest version of "appears everywhere" compatible with the Regulator's requirement.

For D: the WBS (Part A.4) is built from the scope that survives after Part B.6's cuts, not the other way around — capacity is not discovered to be short after the fact.

For E: the verification endpoint's response shape is a first-class design decision (Part C.3), not a filter applied to a general-purpose one.

The named cost, stated once here and carried through the rest of this dossier: the Head of Operations does not get literal retroactive correction of an already-issued verification, and a correction made in the hour before or during results day is not visible on an already-generated payload until the next scheduled refresh. This is treated as the price of Conflicts A and B's resolution, not a separate failure — it is recorded as the lowest-ranked quality attribute in Part B.1, and as the first line of the trade-off register in

Part B.5.

Part B.1 — Quality attributes, ranked

Each attribute is written as a scenario with a number, not a single word. Ranking follows the scenarios; each entry states what the ranking costs.

1. Cost predictability (highest)

Scenario: Monthly infrastructure spend does not exceed NGN 14,000,000, including the results-day peak hour, for the first two years. **Cost of ranking it first:** every other attribute below is architected to fit inside this ceiling rather than the reverse. Specifically, it forces the read path off the live database during results day (Part B.3), which is the single decision most other decisions in this dossier trace back to.

2. Availability during the results-day window

Scenario: The result-check endpoint answers 99.95% of requests during the results-day peak hour, which allows twenty-two seconds of failure in that hour. **Cost:** buys almost all its safety margin from pre-computation before noon, not from real-time scaling — so anything that must be computed live at the moment of the check (a fresh database read, a live permission check against changing data) is a liability we deliberately avoid on this path.

3. Auditability and 50-year reproducibility

Scenario: Any result released to a candidate, school, or employer can be reconstructed exactly as it stood on any date up to 50 years after release, and every amendment names the officer and evidence that produced it. **Cost:** results are never updated in place (Part B.4, ADR-002). This costs storage (every version is kept, not overwritten) and costs the Head of Operations the "instant everywhere" amendment experience they asked for — see Conflict C in this dossier's framing.

4. Accessibility on low-end devices

Scenario: A candidate on a 2G connection with a feature phone, using USSD or SMS with no browser, completes a result check in one session without a data connection. **Cost:** the candidate-facing payload and protocol are designed for the worst device and network in the population, not the average one — richer experiences (result history, certificate download, formatted transcripts) are pushed to the browser channel and explicitly excluded from USSD/SMS (FR-604).

5. Confidentiality of verification responses

Scenario: An employer verifying one certificate by certificate number and date of birth receives only the certificate's validity and the subject grades it covers — never the candidate's other subjects, address, phone number, or examination centre. **Cost:** the verification endpoint cannot simply reuse the candidate's full result record; it needs its own filtered view, maintained and tested separately from every other read path (FR-401, FR-402).

6. Verification throughput growth

Scenario: The verification endpoint sustains a twelve-fold increase in monthly volume, from 40,000 to 500,000, without a corresponding increase in cost or latency. **Cost:** ranked below confidentiality and accessibility because, at 500,000/month, average load is roughly 0.19 requests/second — trivial next to the results-day peak of 2,900/second. The real risk here is not throughput; it is that this channel is a machine client that can retry aggressively, so the cost paid is defensive: rate limiting and connection-reuse design (Part C.1), not extra compute.

7. Freshness of amendment visibility (lowest — deliberately)

Scenario: A result amendment is visible to every new check, download, or verification within one scheduled cache-refresh cycle, not instantly, during the results-day window. **Why it is ranked last, and what that costs:** this is the attribute we have agreed to be worse at. An amendment made in the hour before or during results day will not appear on an already-generated candidate payload or school export until the next refresh cycle runs. The Head of Operations feels this cost directly; it is the price paid for attributes 1 and 2 above, and it is why amendments during the sitting are treated as a controlled, scheduled operation rather than a live one (ADR-001, ADR-002).

Part B.2 — C4 diagrams

Diagrams are submitted as files in the repository (diagrams/*.drawio, editable in diagrams.net / draw.io), per the submission rule that diagrams belong in the repository as files rather than as images pasted into this PDF. This section describes each one and states what it deliberately leaves out.

System Context — `diagrams/context.drawio`

Shows Takarda and every external party it exchanges data with at run time: the Candidate, the School, the Council Officer, the Regulator/Council Board, the Employers' Association, the Payment Partner, and the Telecom Aggregator that carries USSD/SMS traffic.

What it deliberately leaves out: the legacy 2009 system (a one-time migration source, not a running dependency once the platform is live), any internal Council system not touched by this platform (finance, HR), and the author's own build/deploy tooling. A context diagram answers "who does this system talk to," not "everything the Council operates."

Container — `diagrams/container.drawio`

Shows the runnable pieces: the API Gateway/Edge; the Public Read Service (serving both candidate checks and school exports, reflecting ADR-001's decision that these two consumers share the same static-tier design); the PIN & Payment Service and its Reconciliation Job; the Results & Amendment Service; the Verification Service; the Officer Console; the Read-Model Builder; the Message Broker carrying amendment events; and the three data stores — the primary PostgreSQL database, its read replicas, and the object storage/CDN tier that holds pre-materialized results-day payloads. It also shows the concrete mechanism behind Conflict C's resolution (Part B.0): the Verification Service consumes the same amendment events the Read-Model Builder does, and pushes a status-change webhook to the Employers' Association when a certificate it has verified before is superseded (FR-406) — Conflict C is not just a policy statement in Part B.0, it is a drawn data path.

What it deliberately leaves out: the specific deployment topology (containers, VMs, or managed services), the cloud provider, and exact scaling numbers for each service — those are implementation and capacity decisions, not architectural ones, and the brief does not ask for a capacity model beyond what the decisions themselves need.

(Verified: every diagram in this repository was checked programmatically — `scripts/render_drawio_check.py` parses each `.drawio` file's own geometry, renders it, and flags overlapping boxes or edges pointing at a node that doesn't exist. All three currently report zero of either.)

Component — [diagrams/component-read-model.drawio](#)

Zooms into the **Read-Model Builder**, chosen as the part of this system most likely to be got wrong, because it is the exact seam where two decisions that are individually simple — ADR-001 (flat-cost, instant reads via pre-materialization) and ADR-002 (amendments are versioned, never instant) — have to coexist without either silently losing an amendment or publishing a half-built payload. It shows the Refresh Scheduler (drives the pre-release full build and the post-release scheduled ticks), the Amendment Event Consumer and Dirty-Set Tracker (track which candidates/schools need regeneration without rebuilding everything on every amendment), the two generators, a Consistency Checker (a row-count sanity check that exists specifically to catch a partial or corrupt build before it goes live), and the Publisher (an atomic/versioned write, so a reader never sees a half-updated results set).

What it deliberately leaves out: the exact queueing/backoff mechanism between the event consumer and the dirty-set tracker, and the specific refresh-cycle interval — both are operational tuning parameters to be set from real results-day load-test data (WBS package 49), not architectural commitments to make on paper now.

Part B.3 — The architecture pattern

Chosen: staged read/write separation (a CQRS-shaped split) around a conventional service backend

A modestly-sized set of transactional services (PIN/payment, results ingestion, amendments, verification, officer search) write to and read from a single relational primary with read replicas, sized for the platform's real year-round load (verification growing to 500,000/month ~ 0.19/sec average; school/officer access outside results day). Separately, a scheduled read-model builder pre-computes the results-day payloads — per-candidate result documents and per-school export files — and publishes them to object storage behind a CDN before the announced release time. Candidates and schools read from that static tier during the results-day window; nothing on that path touches the primary database. Amendments write to the transactional core as versioned records and are folded into the static tier on the next scheduled refresh, not instantly.

Rejected: single scaled-for-peak monolith

One deployable, one live database, scaled continuously to survive the results-day peak (2,900 checks/second against an eleven-month baseline under 20/second — a ~150x ratio). **Rejected because of that specific ratio**, not because monoliths are generally wrong: provisioning the whole system for a 150x spike that occurs for roughly one hour a year means paying for that capacity for the other 8,759 hours, which breaks the Finance Director's flat NGN 14,000,000/month ceiling directly. A monolith also has no natural seam at which to stop scaling the parts that don't need it (officer search, PIN issuance) independently of the part that does (candidate reads).

Rejected: fully synchronous microservices with no materialized read tier

Splitting the same responsibilities into separately deployable services, but having every read — including the results-day candidate check — call through to the live primary database synchronously, the same way every other read does. **Rejected because it does not touch the actual bottleneck**: the constraint is not organisational (which team owns which service) but numerical (2,900 reads/second against a store sized for under 20/second). Adding service-to-service network hops on that specific path only adds latency and additional failure points to the single request that this system is least allowed to fail, while leaving the flat-cost problem exactly where it was. Microservice decomposition is used here — the transactional core is split by responsibility — but decomposition alone does not solve a read-capacity problem; only removing the results-day read from the live store does.

Does the chosen architecture actually fit NGN 14,000,000/month?

Cloud pricing sheets and capacity models beyond what a decision needs are explicitly not required by the brief, so this is reasoning, not an invoice. The argument for fit is structural, not a hope that a particular vendor happens to be cheap: every rejected option above was rejected specifically because its cost is coupled to **request rate**, and results-day request rate is ~150x the platform's baseline. The chosen design decouples the one thing that spikes (results-day reads) from the two things that cost money (database compute, sized for the ~0.19 req/s year-round B2B load plus routine officer/school traffic; and CDN/object-storage egress, which is priced per byte moved, not per request). The bytes moved on results day are bounded by data size, not traffic: ~1.9 million payloads at roughly 2KB each (Assumption 9) plus 21,000 school export files is on the order of a few gigabytes total for the entire day, regardless of whether every candidate checks in the first minute or spreads across the hour. A design whose cost scales with gigabytes-per-day rather than requests-per-second is the only kind of design that can credibly promise a flat monthly figure under a 150x traffic ratio — which is the property the Finance Director's ceiling actually demands, not a specific number this dossier is positioned to price without vendor quotes.

Part B.4 — Architecture Decision Records

ADR-001 — Pre-materialized static read tier for results-day traffic

Context. Results-day traffic reaches ~2,900 checks/second at peak against an eleven-month baseline under 20/second, while infrastructure cost must stay flat at NGN 14,000,000/month (Conflict A/B).

Decision. Before the announced release time, generate every candidate's result payload and every school's export file and publish them to object storage behind a CDN. Serve all results-day candidate and school reads from that tier; none touch the primary database.

Alternatives. (1) Scale the primary database and application tier to absorb the peak directly — rejected, breaks the flat-cost requirement (Part B.3). (2) Queue requests and drain them gradually after noon — rejected, directly violates the Registrar's "no queue, no waiting page" requirement.

Consequences. Candidate and school reads are cheap and flat-cost at any volume. In exchange, the payload is a snapshot: a correction made in the minutes before release, or during the results-day window itself, is not reflected until the next scheduled refresh (see ADR-002, Part B.6).

One-way door? No. The static tier can be replaced by a live-read design later without invalidating historical data, since it only affects how current data is served, not how it is stored.

Reversal signal. If the measured cost of the static tier plus its refresh jobs exceeds NGN 14,000,000/month for two consecutive results days, or if candidate complaints about stale post-amendment data during the results-day window exceed an agreed threshold set with the Head of Operations, the Engineering Lead reassesses this decision before the next sitting.

ADR-002 — Results are append-only versioned records, never updated in place

Context. The Regulator requires any released result to be reproducible exactly as it stood on any past date for 50 years, with every amendment traceable to an officer and evidence. A mutable "current grade" column cannot satisfy this once overwritten.

Decision. A result's grade is never stored as a single mutable value. Each result is a logical entity (Result) with one or more immutable ResultVersion rows; an amendment creates a new version and links it to the prior version, the acting officer, and the evidence reference, without altering the prior version.

Alternatives. (1) A single mutable result row plus a separate audit log table recording old values — rejected: the audit log becomes the actual source of truth for reproducibility while the "real" table is not authoritative for its own history, which is a fragile split to maintain correctly for 50 years. (2) Full event sourcing of every field on every entity — rejected: the Regulator's requirement is specifically about results and amendments, not about every entity in the system; applying event sourcing everywhere adds complexity the brief's requirements do not ask for.

Consequences. Storage grows with every amendment rather than staying flat (acceptable: amendments run ~475/sitting, a tiny fraction of ~17,000,000 results/sitting). The Head of Operations does not get retroactive rewriting of a verification result an employer already has in hand (Conflict C) — this is the trade this decision makes on their behalf.

One-way door? Yes. Once real candidates' results have accumulated 18+ years of history under this model and that history has been relied on for verification and regulatory purposes, migrating to a different

storage model without breaking continuity of the reproducibility guarantee already extended to the Regulator is not practically reversible.

Reversal signal. This is not expected to be reversed. If it were ever reconsidered, the trigger would be the Regulator formally withdrawing the 50-year reproducibility requirement — the Compliance Lead would be the one watching for that, since no other observation makes this decision worth revisiting.

ADR-003 — PostgreSQL as the primary data store

Context. The workload is relational with strong integrity needs: exact-once PIN/payment handling, atomic linking of a `ResultVersion` to its `Amendment` and officer, and uniqueness constraints across candidates, sittings, and subjects.

Decision. Use PostgreSQL as the transactional primary, with read replicas for year-round read traffic outside the results-day static tier.

Alternatives. (1) A document store (e.g. MongoDB) — rejected: the amendment/versioning integrity model needs multi-row atomic transactions and relational constraints (candidate+sitting+subject uniqueness, foreign-key integrity linking officers to amendments) that a document model supports less naturally, and the Council's grade-count and partial-name queries are exactly the join/aggregate shapes a relational engine is built for. (2) A distributed NewSQL engine (e.g. a globally-distributed SQL cluster) — rejected: actual write volume (~17,000,000 result rows per sitting, twice a year, batch-loaded; ~475 amendments/sitting) is far below the scale that justifies that operational complexity, and the results-day read-scaling problem is already solved by ADR-001, not by the database.

Consequences. A single, well-understood engine that a six-person team can operate confidently within nine months. In exchange, horizontal write scaling is not available if write volume grows by orders of magnitude in the future — not expected given the numbers in this brief.

One-way door? No. A relational engine can, with real migration effort, be swapped for another relational engine later without changing the data model's shape.

Reversal signal. If sustained write throughput to the primary (outside batch-load windows) exceeds 70% of provisioned capacity for three consecutive months, the DBA raises this for reassessment.

ADR-004 — Partition the results archive by examination sitting

Context. The archive will hold ~216,000,000 subject results across 18 years, growing every sitting, and must retain everything for 50 years. Every query in Part D is scoped to a single sitting.

Decision. Partition the `Result/ResultVersion` tables by sitting (year + sitting type). Older partitions can be moved to cheaper storage tiers without touching the active partition.

Alternatives. (1) No partitioning, one large table — rejected: every index on a single 216,000,000-row table grows without bound and every sitting-scoped query (all five in Part D) pays the cost of an ever-larger index even though it only ever needs one sitting's slice. (2) Partition by candidate region/state — rejected: none of the five required query shapes filter by region as their primary access path; this would optimize a query the brief does not ask for (the one query that does group by state, Query 4, is a low-frequency report that does not need this).

Consequences. Sitting-scoped queries stay fast and flat-cost as the archive grows (Part D.3). In exchange, any query that must span many sittings at once (e.g., a candidate's full 18-year transcript) touches multiple partitions — accepted, since no stakeholder in this brief asks for that access pattern at

results-day speed.

One-way door? Yes. Once 18 years of data (24,000,000 candidates, ~216,000,000 results) are relied upon under this partitioning scheme for active verification (growing to 500,000/month) with no permitted downtime, re-partitioning that volume of live, legally-significant data is a major, risky migration, not a routine change.

Reversal signal. If a genuinely new access pattern emerges that must span all sittings for a single candidate at results-day latency (not currently required by any stakeholder here), the Engineering Lead and DBA jointly reassess before committing further sittings to this scheme.

ADR-005 — A distinct protocol per consumer class, not one uniform API

Context. Three consumers have incompatible constraints: a candidate on 2G/feature phone, a school pulling ~400 rows once a year, and the employers' association making up to 500,000 calls/month from a data centre.

Decision. Candidates are served primarily via USSD/SMS through a telecom aggregator, with a lightweight HTTPS/JSON path for those with data access; schools and the employers' association are served via a conventional HTTPS/REST API.

Alternatives. (1) One HTTPS/REST API for all three, with the candidate channel required to have data connectivity — rejected: the Council has explicitly ruled out a design that only serves people with good phones and signal, and a large share of candidates sit the exam in towns served by 2G. (2) One custom binary protocol for all three, optimized for the lowest-capability client — rejected: this would under-serve the employers' association's need for a well-documented, contract-grade REST interface it can integrate against, for no benefit to that consumer.

Consequences. Three integration surfaces to build and operate instead of one, spread across the WBS's Integrations role. In exchange, no consumer is forced onto a protocol mismatched to its constraints — full reasoning in Part C.1.

One-way door? No. Additional channels can be added later (e.g., a mobile app) without displacing the existing three.

Reversal signal. If USSD/SMS usage falls below a small fraction of candidate checks (say, under 5%) for two consecutive sittings, indicating the population has moved to data-capable devices faster than assumed, the Product Lead reassesses whether that channel still earns its operating cost.

ADR-006 — Differentiated authentication per consumer class

Context. The employers' association is a machine client under contract; schools are 21,000 independently-administered accounts; candidates are anonymous one-off users with only a PIN and exam number.

Decision. The employers' association authenticates by mutual TLS client certificate. Schools authenticate by OAuth2 client-credentials bearer token issued at onboarding. Candidates on USSD/SMS are trusted at the level of the telecom aggregator's session and authenticated at the application layer only by PIN + exam number; browser candidates use server-side TLS only, with the same PIN + exam number model.

Alternatives. (1) Bearer tokens for everyone, including the employers' association — rejected: a token can be copied and replayed from any machine, whereas a certificate-pinned mutual TLS connection ties the association's calls to a specific, revocable machine identity appropriate to a contracted B2B relationship.

(2) Mutual TLS for schools as well — rejected: managing certificate issuance, rotation, and revocation for 21,000 independently-administered school accounts is an operational burden disproportionate to the risk; a revocable bearer token is simpler to issue and to cut off per-school if compromised.

Consequences. Three authentication mechanisms to build, document, and operate rather than one. In exchange, each consumer's authentication strength is proportionate to its actual trust relationship with Takarda.

One-way door? No. A consumer class's authentication mechanism can be upgraded later (e.g., moving schools to mutual TLS) without affecting the other two.

Reversal signal. If a school credential compromise incident occurs, the Security Lead reassesses whether bearer tokens remain adequate for that consumer class.

ADR-007 — Reserve/confirm idempotency for PIN purchase, built on Takarda's side

Context. The payment partner charges the card at PIN issuance and accepts no idempotency key; this caused 9,100 double charges last year.

Decision. Takarda creates a PIN record in `PENDING_PAYMENT` state, keyed by a client-supplied idempotency key, before calling the partner. A repeated request with the same key returns the existing record instead of charging again. If the partner's response times out, the PIN is held in `AWAITING_CONFIRMATION` and resolved by an asynchronous reconciliation job, never by re-calling the partner from the client's retry. Full flow in Part C.4.

Alternatives. (1) Wait for the payment partner to add idempotency-key support — rejected: it is stated as not supported today, and the fix must exist for the May sitting; waiting on a third party's roadmap is not a decision Takarda controls. (2) Deduplicate only on the client side (e.g., disable the "buy" button after one tap) — rejected: this does not protect against network-level retries, timeouts, or a candidate reloading the page, which is exactly the failure mode that produced 9,100 double charges.

Consequences. Takarda carries the complexity of a state machine and a reconciliation job that would otherwise sit with the payment partner. In exchange, the double-charge problem is fixed without depending on the partner changing anything.

One-way door? No. If the partner later adds idempotency-key support, Takarda's reserve/confirm layer can be simplified without changing the PIN data model.

Reversal signal. If the payment partner announces native idempotency-key support, the Integrations engineer reassesses whether the reconciliation job can be retired in favour of relying on the partner directly.

ADR-008 — URI-path API versioning with a minimum 12-month support window

Context. Schools integrate with the API infrequently (once a year, around results day) and are the consumer most likely to be running against an old version without realising it.

Decision. Version the API in the URI path (`/v1/...`, `/v2/...`). A version remains supported for a minimum of 12 months after a successor version ships, with `Deprecation` and `Sunset` response headers on the old version once a successor exists.

Alternatives. (1) Header-based versioning (e.g., an Accept media-type parameter) — rejected: it is easy for a school's contracted developer, who touches this integration once a year, to omit or get wrong silently, whereas a URI path is visible in every request they already have working. (2) No formal versioning, evolve the API in place with backward-compatible changes only — rejected: the PIN purchase fix, the DPO field allow-list, and the amendment-visibility model are all changes substantial enough that "backward compatible only" cannot be guaranteed to hold for the life of the platform.

Consequences. Takarda must run and support more than one API version simultaneously for at least 12 months after any breaking change. In exchange, a school that has not re-integrated since last results day keeps working.

One-way door? No. The versioning scheme itself can be changed for a future major version without affecting already-issued versions' behaviour.

Reversal signal. If maintaining two live versions simultaneously ever pushes a results-day load test outside its target margins, the Engineering Lead reassesses the minimum support window length (not the versioning approach itself).

Part B.5 — The trade-off register

One entry per ADR. Every entry names a victim.

#	Decision	Makes better	Makes worse	Who feels it
ADR-001	Pre-materialized static read tier	Results-day read cost and latency, at flat cost	Freshness of results-day reads — a late correction isn't visible until the next refresh	The Head of Operations, and any candidate whose result was corrected in the hour before or during release
ADR-002	Append-only versioned results	Regulator's 50-year reproducibility and amendment traceability	Storage footprint (every version kept); no retroactive rewrite of a result an employer already verified	The Head of Operations, whose "everywhere immediately" request is only partially honoured
ADR-003	PostgreSQL as primary	Team can build and operate it confidently in 9 months; strong fit for join/aggregate query shapes	A single write primary is a single point of write failure — there is no automatic write-failover target, only read replicas	The on-call DBA during the ~17,000,000-row pre-release results load (WBS package 5): if the primary fails mid-load, there is no second writable node to fail over to, only a restore-and-resume
ADR-004	Partition by sitting	Sitting-scoped queries stay flat-cost as the archive grows to 216,000,000 rows	Any query spanning many sittings for one candidate is more expensive	A Council officer fulfilling a candidate's request for a full multi-sitting transcript (a real, if infrequent, request — universities and immigration authorities ask for these) — that query touches every partition the candidate ever sat in, not one
ADR-005	Distinct protocol per consumer	No consumer is forced onto a mismatched protocol; USSD/SMS candidates are genuinely served	Three integration surfaces to build, test, and operate instead of one	The Integrations engineer, who owns all three
ADR-006	Differentiated authentication per consumer	Trust strength matches the actual relationship with each consumer	Three authentication mechanisms to document and operate	The Security Lead and the Integrations engineer, who must keep all three current

#	Decision	Makes better	Makes worse	Who feels it
ADR-007	Reserve/confirm idempotency on Takarda's side	Fixes the 9,100/year double-charge problem without waiting on the payment partner	Takarda carries a state machine and reconciliation job the partner should arguably own	The Integrations engineer, who owns the reconciliation job's ongoing correctness
ADR-008	URI-path versioning, 12-month minimum window	A school that hasn't re-integrated since last year keeps working	Two API versions must run and be tested simultaneously for at least 12 months after any breaking change	The Backend and QA engineers, who carry double the surface area during that window

Part B.6 — What you are not building

1. Real-time push notification of amendments

A reasonable person would expect that when a result is amended, the affected candidate, school, and any employer who verified it are automatically notified. **Left out of Release 1** because it requires a new outbound channel (SMS/email delivery, its own cost and failure handling) that has no stakeholder deadline attached to it in this brief, and would compete for the same nine months as the results-day path, which does have a fixed, unmovable deadline. **Revisit when:** amendment volume or complaint volume about "found out late" exceeds a threshold agreed with the Head of Operations, or when the notification infrastructure being built for other purposes (e.g., a future payment receipt) can be reused at marginal cost.

2. Self-service live analytics dashboards for the Council

A reasonable person would expect the Council to want live, interactive dashboards (grade distributions, pass rates by state, trends over time) rather than the single post-sitting report this design produces (Part D, Query 4). **Left out of Release 1** because Query 4 has no latency requirement stated anywhere in the brief, and building an interactive dashboard layer on top of it would be optimising for a request nobody has made, at the cost of results-day-critical work. **Revisit when:** the Council explicitly asks for continuous, self-service reporting rather than a per-sitting report — at that point this becomes a new, separately-scoped piece of work, not a Release 1 add-on.

3. Any candidate identity check beyond PIN + exam number

A reasonable person would expect stronger proof that the person checking a result is the candidate it belongs to — a photo, a biometric, a second factor. **Left out of Release 1** because the brief describes the existing trust model (PIN + exam number) without flagging it as a known fraud problem, and changing it would touch every channel, including USSD/SMS, where stronger identity checks are hardest to deliver on a feature phone. **Revisit when:** PIN-sharing or impersonation is measured as an actual problem (e.g., via a support-ticket or fraud-complaint volume threshold agreed with the Head of Operations) — not before, since there is no evidence in this brief that it is one today.

4. A self-service dispute/complaints portal beyond the automated double-charge refund

A reasonable person would expect a general-purpose portal for candidates to raise any billing or result dispute themselves. **Left out of Release 1** because the one dispute type with a quantified cost in this brief — the double charge — is fixed structurally by ADR-007 without needing a portal at all; a general dispute portal would be built for problems this brief does not enumerate or size. **Revisit when:** a second dispute type is identified and quantified with real volume, the same way the double-charge problem was quantified at 9,100/year.

Part C.1 — Protocol choice per consumer

Candidate (2G / feature phone / browser)

Chosen: USSD, via a telecom aggregator gateway, as the primary channel; SMS as a fallback for the result content itself; HTTPS/JSON (HTTP/1.1 with keep-alive, TLS 1.3) as a secondary channel for candidates with a data connection.

Why the others lost. A plain HTTPS request over a fresh connection costs, before any application data moves: one TCP handshake (1 round trip) plus one TLS handshake (1 round trip under TLS 1.3's 1-RTT handshake, 2 round trips under TLS 1.2's older negotiation) — 2 round trips for TLS 1.3, 3 for TLS 1.2, before the request itself is even sent. Using this brief's own 2G figure of a ~600ms round trip (estimate, labelled as such — the brief gives packet loss and bandwidth but not RTT directly) as the unit: TCP + TLS 1.3 costs $2 \times 600\text{ms} = 1.2 \text{ seconds}$ of pure connection setup; TCP + TLS 1.2 costs $3 \times 600\text{ms} = 1.8 \text{ seconds}$. Either way, that is over a second spent before the candidate's exam number and PIN have even been transmitted, and it is paid again on every fresh connection unless it is reused. USSD avoids this entirely: it rides the existing signalling channel between the handset and the telecom network that is already established for the phone to be reachable at all, so there is no separate connection to set up. This is the layer at which the decision is actually made — it is a transport-layer and session-layer choice (USSD's own session protocol vs. TCP+TLS), not an application-layer one, and it is decided by the numbers (2G round-trip cost, ~150x results-day spike, "no queue" requirement) rather than by preference.

What breaks if this is wrong. Forcing every candidate onto browser/HTTPS would mean the population described in the brief — "a large share sit the examination in towns served by 2G," many on feature phones — experiences each check as several seconds of connection setup before any result appears, on top of whatever the results-day load itself costs. That directly produces the complaint volume the Registrar's "no queue" requirement is trying to prevent, just moved from a server-side queue to a client-side one the candidate can't see the cause of.

Connection reuse. For the browser/HTTPS path, the server keeps connections alive (HTTP keep-alive) so a candidate's five PIN uses across a session reuse one TLS handshake rather than paying its cost five times, and enables TLS session resumption for a returning candidate.

School (bulk export, ~400 rows/school)

Chosen: HTTPS/REST, HTTP/1.1 or HTTP/2, standard TLS 1.2/1.3.

Why the others lost. A school's admin fetches its export once a year; the connection-setup cost that matters so much for the candidate on 2G is negligible here — a school has ordinary internet connectivity and makes one request, not thousands. USSD would be a poor fit for a 400-row bulk payload, which USSD's page-size limits cannot carry at all. No optimisation beyond a standard REST/TLS setup is justified by the numbers.

What breaks if this is wrong. Over-engineering this path (e.g., a custom binary protocol to save milliseconds) would spend Integrations-engineer time (a scarce resource per the WBS) solving a round-trip-cost problem that does not exist for this consumer.

Employers' association (up to 500,000 verification calls/month, machine-to-machine)

Chosen: HTTPS/REST with HTTP/2 and persistent, pooled connections; mutual TLS (Part C.2).

Why the others lost. At 500,000/month (~0.19/sec average, but arriving from the association's own systems, likely in bursts around business hours or in batches around employer request volume), the cost that matters is not per-request round-trip latency but the overhead of repeatedly establishing new connections from a data-centre client making many calls. HTTP/2 multiplexing over a small pool of long-lived connections amortises the handshake cost across many requests, which is the application-layer and transport-layer decision that matters here — the opposite emphasis from the candidate channel, where the handshake itself had to be eliminated.

What breaks if this is wrong. Forcing the association onto a per-request-new-connection model (no keep-alive, HTTP/1.0-style) would multiply TLS handshake and certificate-verification overhead by 500,000/month for no benefit, and one slow or stalled request on a non-multiplexed connection would queue every request behind it from that same client — exactly the "one slow response blocking the queue behind it" failure mode this decision is chosen to avoid.

Part C.2 — Transport security

TLS termination. TLS terminates at Takarda's edge (the API gateway / CDN in front of the results-day static tier). Traffic between the edge and internal services travels over the private network Takarda controls; it is not re-exposed to the public internet at any internal hop.

Employers' association — certificate-authenticated. The association presents a client certificate issued under the service contract at TLS handshake time; Takarda verifies it against a pinned certificate authority before the request reaches the application layer. This is a machine talking to a machine on a contract with legal accountability behind it, so authentication happens at the transport layer, before any application credential is even read — a compromised or expired certificate is rejected before the request is parsed at all.

Schools — token-authenticated over server TLS. Schools verify Takarda's server certificate (standard TLS) but do not present a client certificate. Instead, each school authenticates at the application layer with an OAuth2 client-credentials bearer token issued at onboarding. Reasoning: 21,000 independently-administered school accounts make certificate issuance, rotation, and revocation at that count an operational burden with no matching benefit — a bearer token is simpler to issue, and just as simple to revoke per school if one is compromised, without touching the other 20,999.

Candidates — server TLS only (browser), carrier-trust boundary (USSD/SMS). A candidate on the browser channel verifies Takarda's server certificate but presents no client credential beyond the PIN and exam number carried in the request body — a person on a phone has no certificate to manage and should not be asked to. On USSD/SMS, there is no end-to-end TLS session between the candidate's handset and Takarda at all; the security boundary is the telecom aggregator's own carrier-grade session, and the candidate is authenticated purely by what they know (PIN + exam number), consistent with the existing trust model this system inherits (Assumption 5).

Why the employers' association and the candidate are not authenticated the same way. The association is a single, known, contracted counterparty capable of managing a certificate and with a direct legal relationship to Takarda if it misuses access; a candidate is one of 1,900,000+ anonymous individuals, most only interacting with the system once or twice a year, many without a device capable of managing a certificate at all. Certificate-based authentication fits the first case and is unworkable for the second.

Part C.3 — The API contract

The full machine-readable contract is in the repository at `api/openapi.yaml` (OpenAPI 3.1, validated with `openapi-spec-validator` — see repository README for how to check it). This section states the design decisions behind it.

Resources, not actions

Every URL names a thing: `/pins`, `/result-checks`, `/schools/{schoolId}/sittings/{sittingId}/results`, `/certificates/{certificateNumber}/verifications`, `/certificates/{certificateNumber}/status`, `/amendments`, `/results/{resultId}`, `/results/{resultId}/versions`. There is no `/checkResult` or `/verifyCertificate` — an action becomes the creation of a resource that represents that event (a "verification," a "result check," an "amendment") rather than a verb bolted onto a noun.

Method, safety, and idempotency, made explicit

Operation	Method	Safe?	Idempotent?	Why
Purchase a PIN	POST / pins	No	Yes, via Idempotency- Key	Creates a resource (has a side effect: a charge), but the idempotency key makes a retried request return the same result rather than charging again — see ADR-007.
Check a result	POST / result-checks	No	Yes, via Idempotency- Key	Decrements a PIN's remaining uses — a real side effect — so it cannot be GET; the idempotency key protects a candidate retrying over a flaky 2G connection from losing a second use for one check.
School export	GET / schools/{schoolId}/sittings/{sittingId}/results	Yes	Yes	Pure read, no state change; repeating it is harmless.
Verify a certificate	POST / certificates/{certificateNumber}/verifications	No	No, by design	Not safe, because it writes a verification-event audit record every time (a real side effect the Data Protection Officer and the Regulator both rely on); deliberately not idempotent, because two verification attempts are two auditable events, not one. The date of birth also travels in the body rather than the query string specifically so it is never logged or cached as part of a URL.

Operation	Method	Safe?	Idempotent?	Why
Record an amendment	POST / amendments	No	Yes, via Idempotency- Key	A real write with regulatory weight; idempotency protects an officer's form double-submit from creating two amendment records for one action.
Read current result / version history	GET / results/{resultId}, GET / results/{resultId}/versions	Yes	Yes	Pure reads.
Check certificate status	GET / certificates/{certificateNumber}/status	Yes	Yes	Pure read of validity state only (ACTIVE/SUPERSEDED) — no date-of-birth challenge, since it returns no subject data at all. This is Conflict C's resolution made concrete (Part B.0): anyone holding a certificate, including years later, can always ask whether it still stands, without Takarda needing to have reached them proactively.

Parameters: filter vs. identify vs. body

- **Path parameters identify a specific resource:** {schoolId}, {sittingId}, {certificateNumber}, {resultId}, {pinId}. These are never optional and never used to filter a collection.
- **Query parameters filter a collection:** e.g. `subject` on the school export endpoint narrows the returned rows without changing which resource is being addressed; `cursor/limit` control pagination. A missing or invalid query parameter never returns 404 — it returns 400 (invalid) or is treated as "no filter" (absent), since it is not identifying anything.
- **Body parameters carry data for a create or check operation:** the PIN's payment method, the result-check's exam number/PIN/sitting, the verification's date of birth, the amendment's reason code and evidence reference. Anything the Data Protection Officer requires never to appear in a URL (date of birth, PIN) is a body field, never a query parameter, so it cannot end up in a server access log or a shared cache key.

Status codes per outcome

Full mapping is in the spec; the pattern is consistent: 200/201 for a normal success, 202 for "accepted, not yet resolved" (a PIN awaiting payment confirmation, a school export still being generated), 400 for a malformed request, 401/403 for missing or insufficient credentials, 404 for a genuinely absent resource, 409 for an idempotency-key conflict (same key, different payload), 422 for a request that is well-formed but cannot be fulfilled as a matter of business rule (a PIN that is exhausted — matching FR-202's exact wording), and 429 for rate limiting. Verification deliberately returns the same status and body shape whether the certificate number does not exist or the date of birth does not match (FR-402), so a caller cannot use the response to enumerate valid certificate numbers.

Versioning

The API is versioned in the URI path (/v1/...), per ADR-008. A version is supported for a minimum of 12 months after a successor version ships; the old version's responses carry `Deprecation` and `Sunset` headers once a successor exists, so a school's integration that has not been touched since last results day continues to work and is told, in-band, that it should eventually move.

Part C.4 — The PIN purchase, fixed

The problem restated

The payment partner charges the card at the moment a PIN is issued and accepts no idempotency key. Any retry — a client timeout, a page reload, a flaky 2G connection — risks a second charge. Last year this happened 9,100 times. The fix has to live entirely on Takarda's side of the boundary, since the partner's behaviour cannot be changed for this release (Assumption 3).

Request flow

1. The client (website, bank terminal, agent app) generates an Idempotency-Key once per purchase attempt and sends it with every retry of that same attempt.
2. Takarda looks up the key. If a PIN record already exists for it, Takarda returns that record's current state immediately and **does not call the payment partner again** — this is the core of the fix, and it does not depend on anything the partner supports.
3. If no record exists, Takarda creates a PIN in state `PENDING_PAYMENT` (uses = 0, inactive) in the same transaction as recording the idempotency key, then calls the payment partner to charge 350,000 kobo.
4. The partner responds in one of three ways: - **Success** -> PIN moves to `ACTIVE`, `remaining_uses` = 5. Response: HTTP 201 with the PIN code. - **Declined** -> PIN moves to `FAILED`. Response: HTTP 402. Nothing was charged; the candidate may retry with a new idempotency key. - **No response within the configured timeout** -> PIN moves to `AWAITING_CONFIRMATION`. Response: HTTP 202 with a status-check URL. Takarda genuinely does not know whether the charge landed, and says so rather than guessing.

State kept

A PIN row (idempotency key, state, remaining_uses, amount_kobo) and a PaymentAttempt row (partner reference if known, outcome, requested_at, resolved_at) per attempt. The idempotency key is unique at the database level, so two near-simultaneous requests with the same key cannot both create a PIN — the second one is rejected by the uniqueness constraint and re-reads the first one's result instead.

Resolving an unknown outcome

A reconciliation job polls the payment partner's transaction-lookup-by-reference call (or, failing that, the next daily settlement file — Assumption 3) for any PIN still in `AWAITING_CONFIRMATION` after 15 minutes, and resolves it to `ACTIVE` or `FAILED` based on what the partner's own record shows. The candidate is never asked to resolve this themselves; the status-check URL from step 4 lets the client poll for the outcome once it is known.

Detecting and refunding a double charge that happens anyway

Even with the above, the partner itself could still charge twice on its own side (a partner-side retry bug, or two genuinely different idempotency keys generated across two separate app sessions by the same candidate). A nightly job matches the payment partner's settlement file against Takarda's PIN ledger, grouping by card token, exact amount (350,000 kobo), and a 30-minute proximity window. Any group with more than one matching charge for what is otherwise the same purchase is flagged automatically; the job calls the partner's refund API for the duplicate charge and records the refund against the PIN record. No spreadsheet, and no human decision, is required for this to happen — matching the Council's own stated cost of the current failure (9,100/year, all currently handled without automation).

Part D.1 — The data model

Entities

- **Candidate** — `candidate_id` (PK), `exam_number` (unique), `surname`, `other_names`, `date_of_birth`, `state`, `exam_centre_id` (FK), `phone`.
- **Sitting** — `sitting_id` (PK), `year`, `sitting_type` (MAY | NOVEMBER).
- **Subject** — `subject_id` (PK), `name`.
- **Result** — `result_id` (PK), `candidate_id` (FK), `sitting_id` (FK), `subject_id` (FK). Unique on (`candidate_id`, `sitting_id`, `subject_id`). This row identifies *that a result exists*; it never itself carries a grade.
- **ResultVersion** — `result_version_id` (PK), `result_id` (FK), `grade`, `version_number`, `effective_from`, `recorded_by_officer_id` (FK, nullable — null only for the original load, non-null for every amendment), `evidence_reference` (nullable — null only for the original load), `is_current` (boolean). **Insert-only**. A Result's current grade is the ResultVersion with `is_current` = true; every prior version stays in the table unchanged, forever.
- **Amendment** — `amendment_id` (PK), `result_id` (FK), `prior_version_id` (FK -> ResultVersion), `new_version_id` (FK -> ResultVersion), `officer_id` (FK), `reason_code`, `evidence_reference`, `amended_at`.
- **Certificate** — `certificate_number` (PK), `candidate_id` (FK), `sitting_id` (FK), `issued_at`, `status` (ACTIVE | SUPERSEDED), `superseded_by_amendment_id` (FK, nullable). (Assumption 8: one certificate per candidate per sitting.) Status flips via FR-505; the certificate row is never deleted, so GET /certificates/{n}/status (FR-405) always has an answer, including for a certificate superseded decades ago.
- **VerificationEvent** — `verification_id` (PK), `certificate_number` (FK), `requester_account_id` (FK), `requested_at`, `match_outcome`, `fields_returned`. Audit-only; never exposed back to the requester beyond the immediate response. Doubles as the subscription list for FR-406: any employer account with a VerificationEvent against a certificate in the trailing 12 months is notified when that certificate's status changes.
- **Pin** — `pin_id` (PK), `idempotency_key` (unique), `amount_kobo` (= 350000), `remaining_uses`, `state`.
- **PaymentAttempt** — `attempt_id` (PK), `pin_id` (FK), `idempotency_key`, `partner_reference` (nullable), `outcome`, `requested_at`, `resolved_at`.
- **ResultCheckEvent** — `check_id` (PK), `pin_id` (FK), `exam_number`, `sitting_id`, `checked_at`, `outcome`. One row per PIN use; supports FR-202's exhaustion rule and fraud/audit review.
- **School, SchoolCandidateEnrollment** (`school_id`, `candidate_id`, `sitting_id`) — scopes the bulk export to a school's own candidates (FR-303).

Cardinality

Candidate 1— Result (9 per sitting, per the brief's average, × however many sittings a candidate sat). Result 1— ResultVersion (>=1, append-only). Amendment references exactly two ResultVersions (prior, new) and one officer — never zero, never more. Candidate 1— Certificate (one per sitting sat). Pin 1— PaymentAttempt (normally one; more only on partner timeout/retry). Pin 1—<=5 ResultCheckEvent.

Money

`amount_kobo` and every payment-related figure is a whole number of minor units (kobo). No monetary value in this model is ever a decimal or floating-point type.

Why this shape, for the Regulator's requirement

A single mutable `grade` column cannot answer "what did this say on 14 March 2019" once it has been overwritten. Here, that question is answered by selecting the `ResultVersion` for a `Result` whose `effective_from` is the latest one at or before the requested date — a query over immutable rows, not a reconstruction from a separate audit log that might drift from the "real" table. **A result row is never updated in place.** The `Result` row is stable identity only; every value it has ever held is a distinct, permanent `ResultVersion` row, and an amendment is the act of creating one more.

Part D.2 — The database architecture

Engine: PostgreSQL. Chosen for this workload because it is fundamentally relational and integrity-heavy: exact-once payment/PIN handling, atomic linking of a `ResultVersion` to its `Amendment` and officer, and uniqueness constraints across candidate/sitting/subject all need real ACID transactions across multiple rows at once. The two reporting-style queries in Part D.3 (grade counts, partial-name search) are also exactly the join/aggregate and extension-index shapes a mature relational engine is built to handle. Full rejection reasoning for two alternatives is in ADR-003.

Replication. One primary, two asynchronous streaming read replicas. **Routing rule:** every write (PIN purchase, result ingestion, amendments, officer actions) goes to the primary. Every read that can tolerate a few seconds of replication lag — school/officer access outside the results-day window, employer verification, the Council's grade-count report, partial-name search — goes to a replica. Results-day candidate and school reads do not go to the database at all; they are served from the pre-materialized static tier (ADR-001), so replica capacity is sized for the platform's year-round load (verification growing to 500,000/month ~ 0.19 requests/second average), not for the results-day peak.

Partitioning. `Result` and `ResultVersion` are partitioned by sitting (`year + sitting_type`). Reasoning, tied to the query shapes below: every one of the five required queries is scoped to a single sitting, so partition pruning lets each of them touch only the relevant slice of the archive regardless of how many other sittings exist. Tied to the 50-year retention requirement: older sittings' partitions can be moved to cheaper, colder storage without touching the currently-active partition, and each partition's own indexes stay bounded in size as the *total* archive grows toward 216,000,000 rows.

Sharding: not used. This system does not shard — everything above lives on one primary (plus its replicas), because the actual write volume (~17,000,000 result rows loaded per sitting, twice a year, in a batch window; ~475 amendments per sitting) is nowhere near the volume that would justify distributing writes across multiple database instances. Partitioning solves the query-shape and retention problem within a single instance; sharding would solve a write-throughput problem this workload does not have, at the cost of cross-shard query complexity for anything (like the Council's report) that needs to look across a sitting's full population.

Part D.3 — Five queries, predicted

All five are scoped to one sitting; the `Result/ResultVersion` partitioning (Part D.2) prunes to that sitting's partition before any of the reasoning below even begins, so "does it get slower as the archive grows to 216,000,000 rows" is answered by the partitioning decision first, and repeated only where a query's answer differs from that default.

Query 1 — Candidate fetching their own 9 results for one sitting, by exam number

- **Index:** unique index on `candidate(exam_number)`; index on `result_version(result_id)` reachable via `result(candidate_id)` within the sitting partition — in practice a composite `(candidate_id)` index local to each `result/result_version` partition.
- **Access method:** unique index seek on `exam_number` to resolve `candidate_id`, then an index seek (not a scan) within the pruned sitting partition on `candidate_id`, filtered to `is_current = true`.
- **Rows examined vs. returned:** ~9 examined, 9 returned. Tight — as it should be for a single-candidate lookup.
- **Scaling:** flat as the archive grows to 216,000,000 rows. Partition pruning plus a per-candidate index means this query only ever touches one sitting's partition and one candidate's rows, regardless of how many other sittings or candidates exist.
- **If too slow:** not expected to be. If it ever were, the fix would be a covering index (add `subject_id`, `grade` to the index) to make it index-only — cost: marginally larger index, negligible against the write volume this table already carries.

Query 2 — School fetching every result for its ~400 candidates, one sitting

- **Index:** `school_candidate_enrollment(school_id, sitting_id)` to get the ~400 `candidate_ids`; `result_version(candidate_id)` (as above) to fetch their results, joined as a single set, not per-candidate.
- **The trap.** The naive approach — loop over the 400 candidate IDs and issue 400 separate point lookups — is the wrong access method for this query, even though each individual lookup is itself efficient. The correct method is one query using `candidate_id IN (...400 ids...)`, letting the planner do a single index scan (an index range/bitmap scan feeding a hash or merge join against the enrollment list) in one pass, not 400 round trips.
- **Rows examined vs. returned:** ~3,600 (400 candidates × 9 subjects) examined and returned — tight, once the set-based access method is used.
- **Scaling:** flat with archive growth (partition pruning to one sitting); the per-school size (~400) is bounded by real school size, not by archive size. **Note:** on results day this query does not touch the live database at all — it is served from the pre-generated per-school export file (ADR-001). This prediction describes the path taken outside the results-day window, when a school re-downloads later in the year.
- **If too slow:** unlikely at this row count; if a school were ever unusually large (say, thousands of candidates), the fix is the same set-based query — the access method already scales with candidate count, not with a fixed per-request cost.

Query 3 — Employer verifying one certificate by certificate number + date of birth, DPO-filtered

- **Index:** unique index on `certificate(certificate_number)`; `candidate` reached via its primary key through the join, so no extra index needed there; `result_version` reached via `candidate_id` (already indexed for Query 1).
- **Access method:** unique index seek on `certificate_number` (one row), a primary-key join to `candidate` to compare the supplied date of birth, and an index seek on `result_version(candidate_id)` for the sitting the certificate covers.
- **Rows examined vs. returned:** ~1 certificate + ~9 results examined; the response returned to the caller is smaller still, since the application layer strips every field the Data Protection Officer has not permitted (address, phone, exam centre, other subjects) — the DPO filtering is an application-layer concern, not something the index or query plan does.
- **Scaling:** flat with archive growth — a unique index seek plus partition pruning by the certificate's own sitting.
- **If too slow:** not expected to be. No change proposed.

Query 4 — Council counting candidates by grade in Mathematics, broken down by state, for one sitting

- **This is the query that does not need to be fast at all.** It is a one-off report run by Council staff after a sitting, not a request on any latency-bound path.
- **Index:** `result_version(sitting_id, subject_id)` to narrow to Mathematics rows within the sitting (~1,900,000 rows, since every candidate sits Mathematics) rather than scanning the other eight subjects' ~15,200,000 rows in the same sitting; no further index is added.
- **Access method:** index range scan on `(sitting_id, subject_id)` narrowing to Mathematics, joined to `candidate.state`, aggregated by `GROUP BY state, grade`.
- **Rows examined vs. returned:** ~1,900,000 examined, a few hundred returned (`state × grade` buckets). A large gap — but this is the *expected*, correct shape for an aggregate query, not a missing-index signal: the aggregate genuinely needs to touch every Mathematics row in the sitting to count it.
- **Scaling:** does not get slower as the *total* archive grows toward 216,000,000 rows, because partition pruning limits it to one sitting's Mathematics rows regardless of how many other sittings exist — but it is, and remains, proportional to that one sitting's own size (~1,900,000), which is fixed by the exam's own scale, not by the archive's age.
- **If it were made faster:** a covering index adding `state` and `grade` to the `(sitting_id, subject_id)` index would make this index-only — deliberately **not done** (see Part D.4): it would add write overhead to every one of ~17,000,000 result rows loaded per sitting for a report with no latency requirement. Run on a read replica, off the results-day path, at minutes-not-milliseconds cost, this query needs no further optimisation.

Query 5 — Officer searching by partial surname, matching anywhere in the name

- **The trap.** A plain B-tree index on `surname` does not help this query at all. A B-tree accelerates a left-anchored prefix match (`surname LIKE 'oko%'`) via a sorted-order range scan, but a pattern with a leading wildcard (`surname LIKE '%oko%'`) has no relationship to the index's sort order — there is no way to seek to "contains oko anywhere" in a structure sorted by "starts with." Without a purpose-built index, this query falls back to a full sequential scan of the candidate table.
- **Index:** a trigram index (PostgreSQL `pg_trgm` extension, GIN index) on `surname` (and `other_names`), which decomposes each name into overlapping 3-character fragments and can be searched for "contains this fragment" efficiently.
- **Access method:** with the trigram index, a bitmap index scan over matching trigrams, rechecked against the actual pattern; without it, a sequential scan of all 24,000,000 candidate rows.
- **Rows examined vs. returned:** with the trigram index, examined rows are roughly proportional to the number of trigram matches for the searched fragment — much smaller than 24,000,000, though still larger than the handful actually returned, since trigram matching is a filter, not an exact match, and every candidate hit needs a final recheck against the literal pattern. Without the index, examined ~ 24,000,000 to return perhaps a handful — a genuine problem signal here, unlike Query 4's aggregate, because this is an interactive tool an officer is sitting in front of.
- **Scaling:** without the trigram index, cost grows linearly with the candidate table's size — a full scan gets worse every year as more candidates are added. With it, cost grows much more slowly (roughly with the number of trigram matches, not the table size), though not perfectly flat, since more candidates means more trigram collisions for common fragments.
- **If too slow even with the index:** for very short or very common fragments (e.g. a 2-letter search matching a huge number of trigrams), the fix is a minimum-fragment-length rule enforced at the application layer (e.g. require at least 3 characters) rather than a database change — the cost of *not* doing this is an officer-facing search that occasionally degrades to something close to the no-index case for the most common short fragments.

Part D.4 — What every index costs

Index	Serves	Write cost	Storage	Effect on pre-release load (1,900,000 candidates × 9 subjects loaded before release)
candidate(exam_number) unique	Query 1 (resolve candidate)	Negligible — candidate rows are inserted in batch twice a year, not on any hot write path	Small (one string/integer key per candidate)	None — the candidate table is not written during the results-day load; it is loaded earlier, at registration
result/result_version(candidate_id) , local to each sitting partition	Queries 1 and 2 (one composite index serves both — see note below)	Real: every one of the ~17,000,000 result rows loaded before a sitting maintains this index at insert time	Moderate — one entry per result row	This is the index actually present during the pre-release load window; it is the cost the brief is pointing at when it asks about "the hours when 1,900,000 results are loaded before release"
certificate(certificate_number) unique	Query 3	Low — certificates are created once per candidate per sitting, a smaller write volume than result rows	Small	Minor — certificate issuance can be scheduled independently of the raw result load
result_version(sitting_id, subject_id)	Query 4	Same load-time window as the candidate_id index above — one more index maintained during the same ~17,000,000-row load	Moderate	Adds to the same pre-release load window's total write cost
pg_trgm GIN trigram index on candidate(surname, other_names)	Query 5	Highest of all five — trigram/GIN index maintenance is heavier per write than a plain B-tree	Largest of all five — trigram indexes are typically several times the size of an equivalent B-tree	Low impact on the <i>results</i> load specifically, since candidate rows are loaded at registration, not at results release — but real impact on registration-time load, twice a year

Note on the first two rows: Queries 1 and 2 can be served by the *same* composite index (candidate_id, local to each sitting partition) rather than two separate ones — Query 1 seeks it for one candidate, Query 2

seeks it for an IN-list of ~400. Counting it once, not twice, avoids double-charging the pre-release load window for something that only needs to be paid for once.

The index deliberately not created

A covering index on `result_version(sitting_id, subject_id)` including `grade` and a `denormalised state` column, which would make Query 4 an index-only scan instead of an index-scan-plus-join. **Declined** because Query 4 has no latency requirement — it is a post-sitting report, not a results-day path — and this index would add write overhead to every one of the ~17,000,000 result rows loaded before each sitting for a query that is already acceptable at its current cost. Being entitled to build it and choosing not to is itself a decision: it keeps the pre-release load window's total index-maintenance cost lower for a query that does not need the improvement.

Part D.5 — What would prove you wrong

Each is a specific, checkable observation — not a feeling that something is slow.

Query 1 (candidate's own 9 results). Wrong if the query plan shows anything other than a single-partition index seek returning ~9 rows — e.g. a sequential scan anywhere in the plan — or if measured p99 latency exceeds **50ms** against production-scale data.

Query 2 (school bulk fetch, ~400 candidates). Wrong if the plan shows 400 separate index probes (a nested-loop, one-row-at-a-time pattern) instead of one set-based scan, or if rows examined exceeds roughly **2x** the expected ~3,600.

Query 3 (certificate verification). Wrong if the plan contains a sequential scan anywhere, or if the returned response ever includes a field outside the Data-Protection-Officer-permitted set (address, phone, exam centre, other subjects) — that second failure mode is a correctness bug, not a performance one, and would be caught by the DPO field-leakage test suite (WBS package 46) before it reached this stage.

Query 4 (Mathematics grade count by state). The claim under test is "this does not need to be fast." Wrong if this report is ever observed taking longer than **5 minutes** on a read replica, or if anyone puts it on a live, user-facing path rather than running it as a scheduled batch report — at that point the "doesn't need to be fast" premise no longer holds and the declined covering index (Part D.4) should be reconsidered.

Query 5 (partial-surname search). Wrong if, with the trigram index in place, the planner still falls back to a sequential scan on `candidate` (visible directly in the query plan), or if measured latency for a typical 4-or-more-character fragment exceeds **1 second**. A slow result for a very short (1–2 character) fragment does *not* falsify the prediction — that case is already named as a known weak point handled by an application-layer minimum-length rule, not a database fix.

Part E — AI usage log

Tool: Claude (Anthropic), used throughout as a drafting and reasoning partner for this dossier, working directly from the assignment brief supplied by the student.

Entry 1 — Stakeholder conflict analysis

Task given: Read the brief's stakeholder list and identify which pairs of stated requirements are in direct conflict, and which single request is impossible as written.

Output: Three conflicting pairs identified — Registrar vs. Finance Director, Head of Schools vs. Finance Director (same root cause: fixed budget vs. a ~150x results-day traffic spike), and Head of Operations vs. Regulator (amendment visibility vs. 50-year point-in-time reproducibility) — with the third also identified as the impossible-as-written request.

Verification performed: Re-checked each conflict against the brief's own stated numbers (NGN 14,000,000/month fixed; <20 checks/sec baseline vs. ~2,900/sec peak; 50-year retention requirement) rather than accepting the framing on first read, to confirm the conflict was forced by the numbers and not an assumption. Held up — no correction needed.

Entry 2 — Index design for the partial-surname search (Query 5)

Task given: Predict the index, access method, and cost for a Council officer's partial-surname search ("matches anywhere in the name") against 24,000,000 candidate rows.

Initial output: The first pass toward this answer treated a standard B-tree index on `surname` as sufficient, reasoning by the general habit that "an index makes lookups fast."

What was actually checked: How a B-tree index actually supports the SQL `LIKE/ILIKE` operator — a B-tree can accelerate a left-anchored prefix match (`'name%'`) via a sorted range scan, because a prefix match aligns with the index's own sort order. A pattern with a leading wildcard (`'%name%'`) has no such alignment; there is no way to seek toward "contains this substring anywhere" in a structure sorted by "starts with."

What was actually true: A plain B-tree index on `surname` gives the officer's "anywhere in the name" search no benefit at all — without a purpose-built index, the query falls back to a full sequential scan of all 24,000,000 candidate rows.

Correction made: Replaced the plain B-tree recommendation with a PostgreSQL `pg_trgm` trigram `GIN` index, which is designed for substring search, and priced its higher write and storage cost separately from the other four proposed indexes in Part D.4. This is recorded in Part D.3 (Query 5) and Part D.4 as the final, corrected answer — the log entry here documents that the first instinct was wrong and why.

Entry 3 — OpenAPI contract validation

Task given: Produce the API contract as a specification file that validates, per the assignment's submission requirement.

Initial output: A spec declared as `openapi: 3.0.3`, using `type: mutualTLS` for the employers' association's security scheme, to represent certificate-based machine authentication (ADR-006).

Verification performed: Ran the file through `openapi-spec-validator` (installed locally) rather than assuming it was correct because it "looked right."

What was actually true: The validator rejected the file — `mutualTLS` is not a valid `securitySchemes` type under OpenAPI 3.0.x; it was only introduced in OpenAPI 3.1.

Correction made: Changed the document version to `openapi: 3.1.0` and re-ran the validator, which then passed with no errors. The validated file is committed at `api/openapi.yaml`.

Entry 4 — Work Breakdown Structure capacity check

Task given: Decompose nine months of work for a six-engineer team into packages of at most two engineer-weeks each, and state honestly whether the total fits.

Output: A package list checked against a raw capacity of 234 engineer-weeks (6 x 39 weeks) and an estimated effective capacity of ~187 engineer-weeks after overhead; totals are re-derived in Part A.4 whenever the package list changes.

Later correction (during refinement): This table initially ran to three pages against the brief's two-page target. The first fix attempted was cutting or coarsening packages to force two pages — rejected, because that would mean either exceeding the two-engineer-week cap per package or quietly dropping real work, both worse than being a page over. The actual fix was typographic (smaller table font and tighter row padding specific to this one table, via a `.wbs` CSS scope in `build_dossier.py`), which reached two pages without changing a single estimate or package boundary.

Further correction (a later refinement pass): A subsequent request asked that no heading be separated from its content by a page break, applied dossier-wide via `-pdf-keep-with-next` on every heading. That pushed the WBS back to three pages: the R0 table alone now fills page 11, so the R1 heading moves to page 12 as a whole unit rather than starting mid-page-11 and letting the table split awkwardly across the break. This was accepted rather than reverted — the heading/content rule was the explicit ask for that pass, and it applies to every heading in the document, not just this one table, so carving out an exception for the WBS would have meant the fix wasn't actually general. Worth recording because it shows the two-page target and the no-orphaned-heading rule can genuinely conflict, and when they do, the correctness rule (stated explicitly, this turn) took priority over the length target (stated implicitly, an earlier turn).

What required verification, and how: This is arithmetic, not a factual claim that could be checked against an external source — verified by recomputing the running totals per phase against the table (Part A.4) rather than trusting a single summed figure. The 20% overhead deduction is explicitly labelled an estimate in the document itself, since no such figure exists in the brief, and no external source was available to check it against.

Entry 5 — Grading a competing submission surfaced a gap in this one

Task given: Independently grade a different student's Takarda submission against the brief, and give a percentage with reasoning.

Output: A findings table and a weighted score (~71%), including a note that the competing submission quantified why a *rejected* cloud architecture (serverless + DynamoDB) blew the budget, but never showed a comparable cost argument for its own *chosen* architecture — an asymmetry: rigor applied to what was rejected, not to what was recommended.

What was actually checked: Whether this dossier had the identical asymmetry. It did — ADR-001 and Part B.3 argued why a scaled-for-peak monolith and synchronous microservices would each break the NGN 14,000,000 ceiling, but never argued why the *chosen* design stays under it, beyond asserting "flat cost by construction."

Correction made: Added a short reasoning paragraph to Part B.3 ("Does the chosen architecture actually fit...") arguing the fit structurally — cost is coupled to data volume (a few GB/day) rather than request rate (the ~150x spike) — rather than inventing specific cloud prices, since the brief explicitly does not require a pricing sheet and fabricated dollar figures would violate the "every number is sourced, measured, or a labelled estimate" rule.

Entry 6 — Reused two ideas from the competing submission, rejected a third

Task given: Take the best parts of the competing submission (Entry 5) and integrate them into this dossier.

What was reused, and why: (1) A sharper three-part proof of *why* the Head of Operations' request is impossible (physical boundary, legal/evidentiary causality, distributed-consistency limits) — integrated into Part B.0, strengthening what was previously a one-sentence assertion. (2) A concrete certificate-status/revocation endpoint with a notification to recent verifiers — integrated as FR-405/FR-406, `GET /certificates/{certificateNumber}/status`, and a `SUPERSEDED` status on the `Certificate` entity, giving Conflict C's resolution an actual mechanism rather than only a policy statement.

What was checked and rejected: The competing submission's own 2G-latency arithmetic ("TCP (1 RTT = 600ms) + TLS 1.3 (1 RTT = 600ms) requires 1.8 seconds") does not add up — $600\text{ms} + 600\text{ms} = 1,200\text{ms}$, not 1,800ms. Rather than reusing the number, Part C.1 was rewritten with the arithmetic shown correctly: TCP + TLS 1.3 is 2 round trips (1.2s at a 600ms RTT), TCP + TLS 1.2 is 3 round trips (1.8s) — the *idea* of putting a concrete number on the handshake cost was worth keeping; the specific number as originally presented was not.

Entry 7 — Closing a self-identified verification gap on the diagrams

Task given: Refine every part of this dossier that scored below full marks in the graded review, including a noted gap that the C4 diagrams were checked for valid XML but never actually rendered or checked for overlapping elements.

What was actually checked: Wrote `scripts/render_drawio_check.py`, a small parser that reads each `.drawio` file's own `mxCell` geometry and style, renders it with `matplotlib`, and programmatically flags any two boxes that overlap or any edge pointing at a node ID that doesn't exist.

What was found, and corrected: The first run of the checker flagged the component diagram's boundary box as "overlapping" every component inside it. Investigating before treating that as a real defect: the checker was ignoring the `verticalAlign=top` style on the boundary's label, so it drew the label centred over the child components instead of at the top of the box, which is not what the actual `style` attribute specifies. The checker was fixed to honour `verticalAlign=top`; re-rendered, the boundary's title sits clear at the top and the reported overlaps were confirmed to be the checker's own earlier bug, not a defect in the diagram. All three diagrams now report zero real overlaps and zero dangling edges — a substantially stronger claim than "the XML parses."

Entry 8 — Rendering a diagram checker prompted checking the dossier's own rendering the same way

Task given: none directly — this followed from the habit Entry 7 established (don't trust that a file "looks right"; render it and look).

What was actually checked: Whether the same discipline applied to the *dossier PDF itself*. It hadn't been — every prior visual check happened to land on pages where lists rendered correctly. Rendering the BRD

and PRD pages specifically (the two documents an earlier grading pass said needed more depth) surfaced a real bug: in both files, a bold lead-in line (**In scope:**, **Explicitly out of scope for Release 1**..., Success is checkable, not aspirational:) was directly followed by a bulleted list with no blank line between them. `python-markdown` treats that as one paragraph and renders the list as run-on dash-separated text instead of actual bullets — the content was always correct, but it was never visually a list.

Correction made: Inserted a blank line between each bold lead-in and its list in `part-a1-brd.md` and `part-a2-prd.md`, then scanned every other dossier file programmatically for the same adjacency pattern (none found) before rebuilding and re-rendering both pages to confirm proper bullets.

Entry 9 — Table text overflowing column borders, and a dead end on the way to fixing it

Task given: Fix two named layout defects: headings separated from their content by a page break, and table text bleeding across column borders (named examples: Part D.4, and Part C.3's "Method, safety, and idempotency" table).

What was checked: Zoomed into the flagged tables at high resolution rather than trusting a normal-resolution screenshot. Part C.3's `/schools/{schoolId}/sittings/{sittingId}/results` and Part D.4's `candidate(exam_number)` were genuinely crossing column borders — long, whitespace-free `<code>` tokens were overflowing because reportlab's default word-wrap only breaks at spaces.

First attempt (rejected): Set `-pdf-word-wrap: CJK` (a reportlab/xhtml2pdf mode that breaks at any character) on every table cell. This did stop the overflow, but re-rendering plain-prose tables (no code, e.g. the trade-off register) showed it was also breaking ordinary English words mid-syllable ("twic/e", "surna/me") wherever a line happened to end — CJK mode does not prefer whitespace breaks over character breaks, it was fixing the bug by degrading every table's readability, not just the ones with the actual problem.

Second attempt (also rejected): Scoped the fix to only `<code>` elements and inserted a zero-width space (U+200B) after `/`, `(`, `,`, `_`, `-` as real break points, keeping normal word-wrap everywhere else. Rebuilding showed every inserted zero-width space rendering as a visible black box — it isn't in the base-14 PDF fonts' encoding (Windows-1252), so reportlab drew a missing-glyph placeholder instead of an invisible break. Checked this the same way the naira symbol and `~` were checked earlier in this project: `chr(0x200B).encode('cp1252')` raises `UnicodeEncodeError`, confirming the character was never going to render correctly in this font, not a one-off rendering glitch.

What was actually done: Replaced the zero-width space with a plain space (always in every font's encoding) after the same separator characters, and scoped the whole transform to run only inside `<td>/<th>` cells (via a regex pass in `build_dossier.py` after markdown conversion), so body prose referencing the same identifiers outside tables is untouched. Re-rendered every table in the dossier: prose tables wrap at natural word boundaries with no mid-word breaks, code-heavy tables (Part C.3, Part D.4) stay within their column borders, verified at high zoom on the previously-worst rows.

Separately, for the heading/content fix: added `-pdf-keep-with-next: true` to `h1/h2/h3` in `build_dossier.py`'s CSS, which is the documented xhtml2pdf mechanism for this exact problem. Confirmed by re-rendering: sections that previously risked a heading landing alone at the bottom of a page now move as a unit to the next page. This cost the WBS a page back (see Entry 4's second correction) — an accepted, explained trade-off, not an oversight.

Appendix — Stated Assumptions

Per the brief's rule ("if you need something that is not in this brief, state your assumption where you use it and carry on"), assumptions are also restated inline wherever they are used. This file collects them in one place.

1. **Country/currency context.** The brief's use of naira and kobo places this in Nigeria; the National Secondary Certificate Council is treated as a fictional body structurally analogous to a national examinations council, and no real-world fact about any actual such body is assumed beyond what the brief states.
2. **PIN ownership model.** A result-checker PIN is sold anonymously through banks, agents and the website and is not tied to a candidate at the point of sale — it is bound to whichever exam number the holder supplies at first use. (*Used in: Data Model, API Contract.*)
3. **Payment partner capabilities.** The partner can charge a card synchronously and returns either a definite success/decline or an indeterminate timeout. It is assumed to expose either a transaction-lookup-by-merchant-reference call or, failing that, a daily settlement file, for reconciliation — the brief states it has no idempotency support but says nothing about lookup or settlement capability. (*Used in: PIN Purchase Fix.*)
4. **Officer authentication.** Council officers who record amendments and run searches authenticate via an internal identity system already operated by the Council; its detailed design is out of scope. (*Used in: Amendment workflow, ADR-006.*)
5. **USSD/SMS delivery.** Candidate access via USSD/SMS is delivered through a telecom aggregator gateway under commercial contract; no specific aggregator is named. (*Used in: Protocol choice, Transport security.*)
6. **Team composition.** Of the six engineers (including the author), the WBS assumes: 3 backend/platform engineers (one doubling as the candidate-channel/USSD specialist), 1 data engineer/DBA, 1 engineer covering integrations (payment partner, employers' association, schools), 1 QA/test engineer. Not specified in the brief.
7. **Legacy system.** The 2009 system is assumed to hold the 18-year archive in some queryable form that can be exported and migrated into the new data model; no detail of its technology is assumed beyond "nobody on staff wrote it."
8. **Certificate identity.** Each candidate receives one certificate per sitting, covering all subjects sat that sitting, identified by a certificate number distinct from the exam number.
9. **Result-check response size.** A single check response (9 subject results plus minimal metadata) is estimated at under 2KB of JSON — small enough that on a 2G link, transmission time is dominated by round-trip/handshake latency, not payload size. Labelled estimate, not measured.
10. **Employers' association integration pattern.** The association's 500,000 monthly verification calls arrive as individual synchronous API calls from its own systems (not a nightly batch file), consistent with "verification calls" in the brief.